



Google Summer of Code

Organization: Learning Unlimited (learningu.org)

Repository: github.com/learning-unlimited/ESP-Website



Learning Unlimited

Regression Tests Expansion for ESP

TABLE OF CONTENTS

1. *About Me*
2. *Executive Summary*
3. *Understanding the Current System*
4. *Deep Problem Analysis*
5. *Proposed Solution — Systematic Regression Coverage Framework*
6. *Implementation Plan*
7. *System Architecture*
8. *Pull Request Plan and Timeline*
9. *Design Safety & Backward Compatibility*
10. *Deliverables*
11. *Stretch Goals and Future Extensions*
12. *Why me?*

SECTION 1: ABOUT ME AND MY CONTRIBUTION

Name: Vipul Subhash Pandey
University: Indian Institute of Information Technology and Management Gwalior
Degree: (IT + MBA) *Integrated 5-years*, Year 3
Email: vipulpandey7917@gmail.com
GitHub: <https://github.com/vipulpandey21>
Timezone: India (GMT + 5:30)

- My open source contributions have primarily been with **Learning Unlimited's ESP codebase**.
- My **Entire contribution in Learning Unlimited** before and after the proposal submission can be tracked [here](#)

Created Issues:

<https://github.com/learning-unlimited/ESP-Website/issues/4337>
<https://github.com/learning-unlimited/ESP-Website/issues/4898>

Assigned Issues :

<https://github.com/learning-unlimited/ESP-Website/issues/1843>
<https://github.com/learning-unlimited/ESP-Website/issues/1835>
<https://github.com/learning-unlimited/ESP-Website/issues/3276>

Pull Requests :

<https://github.com/learning-unlimited/ESP-Website/pull/4371>
<https://github.com/learning-unlimited/ESP-Website/pull/4900>
<https://github.com/learning-unlimited/ESP-Website/pull/4994>

- ★ During codebase exploration, I traced critical execution flows such as:
`get_custom_fields()` → `TeacherClassRegForm. init ()` → `set_class_data()`
→ `ClassSubject.custom_form_data` → `teachersbyFOO()`, and identified that while data propagation is complete, key integration points (especially exports) are untested.
- ★ I also analyzed high-risk modules like `lottery.py` (read in full), `classreg.py`, and `programprintables.py`, identifying areas with complex logic but little to no test coverage.
- ★ This proposal is grounded in these observations — focusing not just on increasing coverage, but on systematically testing critical logic paths, invariants, and edge cases that directly impact correctness.

SECTION 2: EXECUTIVE SUMMARY

The core problem in one sentence:

While reading classreg.py I found a production bug with no test:

- `elif k not in (...) and k[:8] is not 'section_': cls.__dict__[k] = v`
- This uses identity comparison, not value comparison. In Python 3, sliced strings are not guaranteed to be interned — `k[:8]` is not `'section_'` can evaluate to `True` even when `k` starts with `'section_'`, silently writing form fields to `cls.__dict__` instead of `custom_form_data`. The HTTP tests all pass (they check status 200, not data routing). The data is silently wrong.

Beyond this bug:

- the ESP codebase has 15,035 missed lines at 69.91% overall coverage, but the distribution is deeply uneven.
- The highest-risk components have the lowest coverage.
- The Codecov threshold is 0.1% — a PR that deletes 100 lines of tested code and adds nothing passes Codecov. That is not enforcement, it is theater.

This proposal directly addresses:

- Issue #3780 ("Expand test coverage" — parent issue) -
- Issue #3773 ("Tests for program modules") -
- Issues #599, #794, #1452, #933, #3457 (specific untested subsystems)

What This Proposal Delivers:

- Component 0 — Shared Test Infrastructure
 - Factories and a `ModuleHandlerTestMixin` that reduce handler test setup from 8-10 lines to 2-3 lines — removing the friction that is the reason 0%-coverage handlers have no tests today.
- Component 1 — ClassCreationController Unit Tests
 - Direct unit tests for `set_class_data()`, `teacher_has_time()`, and `update_class_sections()` — including a regression test for the is not bug that will FAIL on the current code, confirming the bug exists.
- Component 2 — LotteryAssignmentController Tests
 - First tests ever for `lottery.py`: 1000+ lines, 20+ methods, 0 tests.
 - The lottery runs once per program cycle and is irreversible.
- Component 3 — Module Handler Behavioral Tests

- Behavioral tests for five handlers at 0% coverage: finaidapprovemodule, adminreviewapps, lotterystudentregmodule, onsitecheckinmodule.
- Component 4 — Student Registration Invariant Tests
 - Conflict detection, capacity enforcement, waitlist promotion, grade filtering — invariants with no regression protection today.
- Component 5 — Codecov Threshold Update
 - Raise threshold from 0.1% to 70% with patch coverage at 60%.

Design Principles:

- All new code is test code only — no production logic changes
- All tests are additive, isolated, and xdist-safe
- No migrations, no schema changes, no behavior changes
- Selenium tests are opt-in (--selenium flag), skipped by default

SECTION 3: MY UNDERSTANDING OF THE CODEBASE

- My proposed solution didn't start from the issues, it was started from the code and worked outward.

3.1 The Registration Controller — Tested at the HTTP Layer, Not the Business Layer

- When a teacher submits the class registration form, the request flows through:
 - `TeacherClassRegForm.__init__()` -> `ClassCreationController.makeaclass()` -> `get_custom_fields()` -> `set_class_data()` -> `update_class_sections()` -> `associate_teacher_with_class()` -> `send_class_mail_to_directors()`
- `test_class_creation.py` covers the HTTP layer: `makeaclass` GET/POST, `editclass`, `coteacher` add/remove, form validation. But `ClassCreationController` itself — 280 lines, 16+ methods — has zero direct unit tests. `set_class_data()`, `teacher_has_time()`, `require_teacher_has_time()` contain the business rules.
- The most critical gap I found while reading `classreg.py` is in `set_class_data()`:
 - `elif k not in ('category', ...)` and `k[:8]` is not `'section_': cls.__dict__[k] = v`
- This uses ``is not`` — identity comparison, not value comparison. In Python 3, string interning is not guaranteed for sliced strings. ``k[:8]` is not `'section_'` can evaluate to `True` even when `k` starts with `'section_'`, causing form fields that should be excluded to be written directly to `cls.__dict__`. This is a production bug. There is no test that would catch it.

3.2 The Lottery Controller — 1000+ Lines, Zero Tests

- The lottery runs once per program cycle and is irreversible. A regression in `compute_assignments()` or `fill_section()` cannot be corrected after it runs.
- `lottery.py` contains `LotteryAssignmentController`: 1000+ lines, 20+ methods, numpy-based probabilistic class assignment. It has no test file. Not a single test. The methods that matter most:
 - `compute_assignments()`: core algorithm — calls `fill_section()` for every section in random order, grouped by duration. No test.
 - `fill_section()`: filters students by grade, timeslot availability, capacity, lunch constraints, and section overlap. No test.
 - `check_assignments()`: asserts no section is overfilled and no student's schedule is inconsistent. No test.
 - `save_assignments()`: bulk-creates `StudentRegistration` objects and updates Mailman lists. No test.

- `initialize()`: populates 15+ numpy arrays from the database. No test.

requirements.txt pins numpy==1.16.6. Any upgrade altering random seed behavior or array semantics could silently break grade filtering — e.g., ``possible_students *=(self.student_grades >= section_grade_min[si])`` depends directly on numpy behavior. No test currently guards this.

3.3 The Module Handler Gap — 0% Coverage on High-Consequence Handlers

- `finaidapprovemodule.py`: The `finaidapprove()` view processes POST requests to approve financial aid grants. It calls `req.approve(dollar_amount, discount_percent)` for each selected user. If the amount parsing logic breaks, grants are created with wrong amounts. Coverage: 0%.
- `adminreviewapps.py`: `accept_student()` and `reject_student()` create and expire `StudentRegistration` objects with `relationship='Accepted'`. If these break, students are incorrectly admitted or rejected. Coverage: 0%
- `lotterystudentregmodule.py`: `lotterystudentreg()` serves the student lottery registration page. `timeslots_json()` serves timeslot data for the JavaScript lottery interface. Coverage: 0% behavioral tests (existence check only).

A 0%-coverage module is a module where any change is a regression risk.

3.4 The Test Infrastructure Gap — Friction Is The Reason Tests Don't Exist

- The `setup_roles()` pattern is copy-pasted into at least 8 test files.
- Writing a test for any module handler requires: reading `ProgramFrameworkTest` to understand `setUp` parameters, `copying setup_roles()`, figuring out how to build the module URL, logging in as admin, and making the POST request.
- This takes 30-60 minutes before writing a single assertion.
- That friction is the reason `finaidapprovemodule.py` has 0% coverage despite being used on every program day

3.5 THE CODECOV THRESHOLD — Enforcement In Name Only

- The current threshold is 0.1% (`codecov.yml`, line 5).
- A PR that deletes 100 lines of tested code and adds nothing passes Codecov.
- Patch coverage is disabled entirely (`patch: no`). This provides no enforcement against coverage regression after GSoC ends.

SECTION 4 : PROBLEM ANALYSIS

PROBLEM 1: ClassCreationController Has No Unit Tests (Issues #3780, #3773)

- Root cause:
 - `ClassCreationController` has 16+ methods across 280 lines. None have direct unit tests.
 - The is not identity comparison in `set class data()` is the clearest symptom — a production bug with no test that would catch it.
- Impact:
 - A developer optimizes `getTaughtTime()` for performance. All HTTP tests pass (they check status 200, not business rules).
 - Teachers can now register unlimited classes in a 6-hour program. The bug is in `require teacher has time()` — a method with 0 unit tests.

PROBLEM 2: LotteryAssignmentController Has Zero Tests (Issues #3780, #794)

- Root cause:
 - `lottery.py` is 1000+ lines, 20+ methods, 0 tests. The lottery runs once per program cycle and is irreversible.
- Impact:
 - A developer upgrades numpy to fix a security vulnerability. The random seed behavior changes. `fill_section()` now assigns some students to classes outside their grade range.
 - No test catches this. The chapter discovers it when parents call about their 7th-grader in a 10th-grade class.

PROBLEM 3: Five Module Handlers Have 0% Coverage (Issue #3773)

- Root cause:
 - `finaidapprovemodule.py` and `adminreviewapps.py` have 0% coverage.
 - These are not obscure utility functions — they are the views admins use on program day to approve financial aid and accept students into classes.
- Impact:
 - `finaidapprovemodule.py` processes POST requests with `userchecklist`, `approve blanks`, `amount max dec`, and percent parameters.
 - If `amount max dec` is passed as a string instead of Decimal, grants are created with wrong amounts. No test catches this.

PROBLEM 4: Student Registration Invariants Have No Regression Protection (Issues #3780, #599)

- Root cause:
 - The invariants that must always hold — no timeslot conflicts, no overcapacity enrollment, correct waitlist promotion — have no dedicated regression tests.
 - They are exercised incidentally by HTTP tests but never directly asserted.
- Impact:
 - A developer fixes a bug in the scheduling module and accidentally removes a conflict check. Students register for two classes at the same time.
 - On program day, 40 students show up to classes that are already full. No test caught this before it reached production.

PROBLEM 5: The Codecov Threshold Provides No Enforcement

- Root cause:
 - threshold is 0.1% and patch coverage is disabled.
 - Any PR passes regardless of coverage impact.
- Impact:
 - Coverage debt accumulates silently after GSoC ends. T
 - the work done in this proposal can be undone by future PRs with no CI enforcement.

(Problems 1–4 are fully addressed. Problem 5 is addressed by raising the threshold to 70% with patch coverage at 60%)

SECTION 5 : PROPOSED SOLUTION:

(SYSTEMATIC REGRESSION COVERAGE FRAMEWORK)

- 5 components, *ordered by risk* and alignment with the mandatory GitHub issues.

COMPONENT 0: Shared Test Infrastructure (LOW EFFORT, HIGH LEVERAGE)

- THE GAP (from Section 3.4):
 - The `_setup_roles()` pattern is copy-pasted into 8+ test files. Writing a handler test takes 30-60 minutes of boilerplate before the first assertion. That friction is why 0%-coverage handlers have no tests.
- THE FIX:
 - Add `esp/tests/factories.py` with `make_user(role)`, `make_program(**kwargs)`, `make_class(program, teacher)` — replacing the copy-pasted setup pattern.
 - Add `esp/program/modules/tests/support/mixins.py` with `ModuleHandlerTestMixin: login_as(role), get_module_url(handler, view), assert_view_ok(url), assert_view_forbidden(url, user), post_to_module(handler, view, data)`.
 - This reduces module handler test setup from 8-10 lines to 2-3 lines — permanently, for every future contributor, not just for this GSoC project.
- WHY THIS IS THE RIGHT APPROACH:
 - The friction of writing tests is the reason they don't exist. Removing that friction is more valuable than writing tests that future contributors can't easily extend. This is infrastructure, not just convenience.

COMPONENT 1: ClassCreationController Unit Tests (Issues #3780, #3773)

- THE GAP (from Section 3.1): `ClassCreationController` has 16+ methods, 0 direct unit tests. The is not identity bug in `set_class_data()` has no test that would catch it.

- THE FIX:
 - Add test_classreg_controller.py with four test classes:
 - GetCustomFieldsTest: Tag not set → empty OrderedDict; Tag set → correct fields returned; Form.DoesNotExist → empty dict.
 - SetClassDataTest: standard fields route to cls.__dict__; custom fields route to custom_form_data; explicit regression test for the is not identity bug — this test will FAIL on the current code, confirming the bug exists, and proposes the one-character fix (is not → !=).
 - TeacherTimeTest: teacher_has_time() returns True/False correctly; require_teacher_has_time() raises ESPError when over capacity, does not raise when under.
 - UpdateClassSectionsTest: num_sections=3 creates 3 sections; reducing from 3 to 1 deletes 2; all sections get cls.duration after update.
 - These tests call controller methods directly, not through the HTTP layer. Together with test_class_creation.py (HTTP layer), they provide defense in depth: a regression in the controller is caught even if the view returns 200.

COMPONENT 2: LotteryAssignmentController Tests (Issues #3780, #794)

- THE GAP (from Section 3.2):
 - The lottery runs once per program cycle and is irreversible. lottery.py has 1000+ lines, 20+ methods, 0 tests.
- THE FIX:
 - Add test_lottery.py with four test classes:
 - LotteryBasicTest: compute_assignments() on a program with students and StudentSubjectInterest records completes without exception; every interested student is either enrolled or waitlisted after the lottery.
 - LotteryInvariantTest: no student has two enrolled classes at the same timeslot; no section has more enrolled students than capacity (mirrors check_assignments() assertion); no student enrolled in class outside grade range when check_grade=True.

- *LotteryEdgeCaseTest*: lottery on a program with no StudentSubjectInterest records completes without error; given the same numpy random seed, compute_assignments() produces the same student_sections array — regression guard for numpy upgrades.
 - *LotterySaveTest*: save_assignments() creates StudentRegistration objects with relationship='Enrolled' for all assigned students; clear_saved_assignments() expires existing Enrolled registrations without deleting them.
- These tests verify the hard invariants — no conflicts, no overcapacity, no grade violations — that must hold regardless of the random seed. They do not attempt to verify the probabilistic fairness of the algorithm.
- WHY THIS APPROACH:
 - compute_assignments() modifies only numpy arrays in memory. No DB writes until save_assignments() is called. LotteryInvariantTest can verify all invariants without touching the database, making it fast and xdist-safe.

COMPONENT 3: Module Handler Behavioral Tests (Issue #3773)

- THE GAP (from Section 3.3):
 - finaidapprovemodule.py, adminreviewapps.py, lotterystudentregmodule.py have 0% behavioral coverage. These are used on every program day.
- THE FIX:
 - *test finaidapprove.py*: GET returns form with requests in context; POST with user IDs creates FinancialAidGrant with correct amount; POST without approve_blanks skips users with blank household_income; non-admin GET → 403.
 - *test adminreviewapps.py*: accept_student() creates StudentRegistration with relationship='Accepted'; reject_student() expires it; non-admin GET → 403.
 - *test lotterystudentregmodule.py*: timeslots_json() returns correct [id, short_description] pairs; unauthenticated GET redirects; student with Priority/1 registration sees it on viewlotteryprefs.
 - *Extend onsitecheckinmodule.py tests*: POST with valid student ID creates Record with event='attended'; POST with nonexistent ID returns JSON error; non-admin POST → 403.
 - All use ModuleHandlerTestMixin from Component 0.

COMPONENT 4: Student Registration Invariant Tests (Issues #3780, #599)

- THE GAP (from Section 4, Problem 4):
 - Conflict detection, capacity enforcement, waitlist promotion, and grade filtering have no dedicated regression tests.
- THE FIX:
 - Add test_studentreg_controller.py with four test classes:
 - EnrollmentConflictTest: student registers for class A → enrolled; student tries class B at same timeslot → conflict detected, not enrolled in B.
 - CapacityEnforcementTest: class at class_size_max → new registration rejected or waitlisted; class under capacity → registration accepted.
 - WaitlistPromotionTest: student A drops full class → first waitlisted student B is promoted to enrolled.
 - GradeFilterTest: student in grade range → enrollment succeeds; student outside grade range → blocked.

COMPONENT 5: Codecov Threshold Update

- THE GAP (from Section 3.5):
 - threshold: 0.1% and patch: no means no enforcement.
- THE FIX:
 - Modify codecov.yml: threshold 0.1% → 70%, patch coverage enabled at 60%. 70% is below the current 69.91%, so this does not immediately fail any existing PR — it only prevents future drops. 60% patch coverage means new code added in any PR must be at least 60% tested to merge.

SECTION 6 :IMPLEMENTATION PLAN

(CODE-LEVEL)

(Complete file-function change map. Each entry states what is added, why it is safe, and what is intentionally left unchanged)

6.0 Component 0 — Shared Test Infrastructure

- ADD: esp/tests/factories.py
 - make_user(role): creates ESPUser + Group membership in 1 call
 - make_program(**kwargs): creates Program with sensible defaults
 - make_class(program, teacher): creates ClassSubject + ClassSection + category
 - WHY SAFE: Purely additive. No existing test is modified.
- ADD: esp/program/modules/tests/support/mixins.py
 - ModuleHandlerTestMixin:
 - login_as(role): self.client.login() + role assertion
 - get_module_url(handler, view): builds /learn|teach|manage/<prog>/<view>
 - assert_view_ok(url): GET -> assertEquals(200)
 - assert_view_forbidden(url, user): GET as user -> assertIn([302, 403])
 - post_to_module(handler, view, data): POST + return response
 - WHY SAFE: Purely additive. Existing tests do not use this mixin.

6.1 Component 1 — ClassCreationController Unit Tests

- esp/program/modules/tests/test_classreg_controller.py [NEW]
 - Uses CacheFlushTestCase (rotates CACHE_PREFIX so Tag.getProgramTag() never returns stale values across test cases).
 - GetCustomFieldsTest:
 - Tag not set → get_custom_fields(program=prog) returns empty OrderedDict.
 - Tag set to valid form ID → result contains 'cf_<id>' keys with correct labels and field types.
 - Form.DoesNotExist → empty dict, no exception.
 - SetClassDataTest (ProgramFrameworkTest):
 - Standard fields (title, description) route to cls.__dict__ correctly. -
 - Custom field keys route to custom_form_data, not cls.__dict__. -
 - test_is_not_bug_regression(): passes a field key starting with 'section_' to set_class_data() and asserts it does NOT appear in

cls.__dict__. This test FAILS on the current code — confirming the bug — and the PR proposes the one-character fix (is not → !=). Whether to include the fix in the same PR or a separate PR will be decided with mentors during community bonding. If deferred, the test is marked xfail with a comment.

- *TeacherTimeTest (ProgramFrameworkTest):* -
 - teacher_has_time() returns True when teacher has available hours.
 - teacher_has_time() returns False when teacher is at program capacity.
 - require_teacher_has_time() raises ESPError when over capacity. -
 - require_teacher_has_time() does not raise when under capacity. ‘
- *UpdateClassSectionsTest (ProgramFrameworkTest):*
 - num_sections=3 → cls.sections.count() == 3 after update. -
 - Reducing from 3 to 1 → section_list[1:].delete() called, count == 1. -
 - All sections get cls.duration after update. INTENTIONALLY UNCHANGED: set_class_data() and update_class_sections() in classreg.py — tests call these methods directly, no production code changes.

6.2 Component 2 — Lottery Controller Tests

- esp/program/tests/test_lottery.py [NEW]
 - All classes use ProgramFrameworkTest with num_students=10, num_teachers=3, classes_per_teacher=2, num_timeslots=3. setUp() calls add_student_profiles() and creates StudentSubjectInterest records for each student.
 - *LotteryBasicTest:*
 - compute_assignments() completes without exception.
 - Every student with a StudentSubjectInterest is either enrolled or waitlisted after the lottery.
 - *LotteryInvariantTest:*
 - No student has two enrolled classes at the same timeslot (checked via student_schedules numpy array — no row has two True entries in the same timeslot column).
 - numpy.sum(student_sections, 0) <= section_capacities for all sections.
 - student_grades[i] >= section_grade_min[j] for all (i,j) where student_sections[i,j] is True, when check_grade=True.
 - *LotteryEdgeCaseTest (CacheFlushTestCase):*

- Program with no StudentSubjectInterest → initialize() sets all arrays to zero → compute_assignments() completes, no StudentRegistration created.
- numpy.random.seed(42) → compute_assignments() twice → student_sections arrays are identical. Documented as numpy-version-specific.
- LotterySaveTest:
 - save_assignments(StudentRegistration.objects.bulk_create() with relationship='Enrolled' for all assigned students; count matches numpy.sum(student_sections).
 - clear_saved_assignments() → end_date set to now() → valid_objects() returns o.
- WHY SAFE: compute_assignments() modifies only numpy arrays in memory. No DB writes until save_assignments() is called. LotterySaveTest runs inside a transaction rolled back after the test.

6.3 Component 3 — Module Handler Behavioral Tests

- esp/program/modules/tests/test_finaidapprove.py [NEW]
 - ◆ FinAidApproveTest (ProgramFrameworkTest+ModuleHandlerTestMixin):
 - GET /manage/<prog>/finaidapprove → 200, 'requests' in context.
 - POST userchecklist=[user.id], amount_max_dec='25.00' → req.approve() called → FinancialAidGrant.objects.filter(request=req).exists() == True. -
 - POST without approve_blanks, req.household_income=" → req.approve() NOT called → no FinancialAidGrant created. -
 - Student GET → 403.
- esp/program/modules/tests/test_adminreviewapps.py [NEW]
 - ◆ AdminReviewAppsTest (ProgramFrameworkTest + ModuleHandlerTestMixin):
 - GET /manage/<prog>/accept_student?cls=<id>&student=<id> → StudentRegistration with relationship='Accepted' created. -
 - GET /manage/<prog>/reject_student → reg.expire() → valid_objects() returns o. -
 - Student GET → 403.
- esp/program/modules/tests/test_lotterystudentregmodule.py [NEW]
 - ◆ LotteryStudentRegModuleTest (ProgramFrameworkTest): -
 - GET /learn/<prog>/timeslots_json → JSON array of [id, short_description] pairs, ordered by start time, matching program.getTimeSlotList(). -

- Unauthenticated GET /learn/<prog>/lotterystudentreg → redirect.
 - Student with StudentRegistration(relationship='Priority/1') → GET viewlotteryprefs → response contains that registration.
- MODIFY: esp/program/modules/tests/onsitecheckinmodule.py
- ◆ Add three behavioral tests using ModuleHandlerTestMixin:
 - POST /onsite/<prog>/ajaxbarcodecheckin {barcode: student.id} → Record.objects.create(event='attended', user=student) → JSON success.
 - POST with nonexistent ID → no Record created → JSON error key.
 - Non-admin POST → 403.

6.4 Component 4 — Student Registration Invariant Tests

- esp/program/modules/tests/test_studentreg_controller.py [NEW]
- ◆ All classes use ProgramFrameworkTest with add_student_profiles() and schedule_randomly().
 - ◆ EnrollmentConflictTest:
 - sec.preregister_student(student) → StudentRegistration created with relationship='Enrolled'.
 - Student enrolled in sec_A at timeslot T → preregister_student for sec_B at same timeslot T → conflict detected, not enrolled.
 - ◆ CapacityEnforcementTest:
 - section.num_students() == section.capacity → additional preregister_student() rejected or placed on waitlist.
 - section.num_students() < section.capacity → registration accepted.
 - ◆ WaitlistPromotionTest:
 - Full section, student B waitlisted → student A drops → StudentRegistration(A) expired → student B promoted to Enrolled.
 - ◆ GradeFilterTest:
 - student.grade in [section.grade_min, section.grade_max] → needs_student_in_grade passes → enrollment succeeds.
 - student.grade outside range → needs_student_in_grade blocks → rejected.

6.5 Component 5 — Codecov Configuration

- MODIFY: codecov.yml
- ◆ threshold: 0.1% → 70%
 - ◆ patch coverage: enabled at 60% for new code in PRs.
 - ◆ WHY SAFE: 70% is below the current 69.91%, so this does not immediately fail any existing PR. It only prevents future drops.

SECTION 7 : SYSTEM ARCHITECTURE

(No new tables. No new columns. No migrations. All new code is test code only. One configuration file is updated: codecov.yml.)

7.1 Test Layer Map

Layer	Base Class	Speed	xdist-safe	New additions
Unit	CacheFlushTestCase	<1s	Yes	GetCustomFieldsTest, SetClassDataTest, TeacherTimeTest, LotteryEdgeCaseTest
Integration	ProgramFrameworkTest	2-5s	Yes	LotteryBasicTest, LotteryInvariantTest, LotterySaveTest, all handler tests, all reg invariant tests
End-to-End	StaticLiveServerTestCase	10-30s	No (opt-in)	TeacherClassRegSelenium, StudentClassRegSelenium

(For text alignment issue and to reduce space use, i have used other app for this table)

7.2 CI Integration

- The existing CI pipeline runs `pytest -n auto --cov=. --cov-report=xml`. All new unit and integration tests are automatically picked up. No CI changes are needed. The `--selenium` flag is not passed in CI by default, so Selenium tests do not slow down the standard run. The `codecov.yml` threshold update takes effect immediately after merging.

7.3 xdist Compatibility

- All new tests use Django's TestCase (via CacheFlushTestCase or ProgramFrameworkTest), which wraps each test in a transaction rolled back after the test. No test writes to the filesystem or uses global state. LotteryAssignmentController.compute_assignments() modifies only numpy arrays in memory no DB writes until save_assignments() is called.
- ESTIMATED COVERAGE IMPACT:
- Overall: 69.91% ? 71-72%
- esp/program: 68.05% ? 71-73%
- lottery.py: ~0% ? 40-60%

SECTION 8 — PULL REQUEST EXECUTION PLAN AND TIMELINE

Each PR is small, independent, and review-friendly. PRs will be submitted early in the week. Mentor feedback will be incorporated within 24-48 hours.

- **PR1: Shared Test Infrastructure (Component 0)**
 - Files: esp/tests/factories.py, esp/program/modules/tests/support/mixins.py
 - Delivers: make_user(), make_program(), make_class(), ModuleHandlerTestMixin
 - Dependencies: None
 - Size: ~60 lines
- **PR2: ClassCreationController Unit Tests (Component 1)**
 - Files: esp/program/modules/tests/test_classreg_controller.py
 - Delivers: GetCustomFieldsTest, SetClassDataTest (incl. is not bug regression), TeacherTimeTest, UpdateClassSectionsTest — 13 tests total
 - Dependencies: PR1 (factories)
 - Size: ~200 lines of tests
- **PR3: Lottery Controller Tests (Component 2)**
 - Files: esp/program/tests/test_lottery.py
 - Delivers: LotteryBasicTest, LotteryInvariantTest, LotteryEdgeCaseTest, LotterySaveTest — first tests ever for LotteryAssignmentController
 - Dependencies: PR1 (factories)
 - Size: ~150 lines of tests
- **PR4: Module Handler Behavioral Tests (Component 3)**
 - Files: test_finaidapprove.py, test_adminreviewapps.py, test_lotterystudentregmodule.py, onsitecheckinmodule.py (modified)
 - Delivers: Behavioral coverage for 4 handlers at 0%
 - Dependencies: PR1 (ModuleHandlerTestMixin)
 - Size: ~180 lines of tests
- **PR5: Student Registration Invariant Tests (Component 4)**
 - Files: esp/program/modules/tests/test_studentreg_controller.py
 - Delivers: EnrollmentConflictTest, CapacityEnforcementTest, WaitlistPromotionTest, GradeFilterTest
 - Dependencies: PR1 (factories)
 - Size: ~120 lines of tests
- **PR6: Codecov Configuration + Documentation**

- Files: codecov.yml, docs/dev/testing.rst
- Delivers: threshold 0.1% → 70%, patch coverage at 60%, testing guide
- Dependencies: PR2-PR5 merged (coverage must be at 70%+ before raising bar)
- Size: 3 lines of config, ~50 lines of docs

TIMELINE MAPPING:

- **Community Bonding (May 1-26):**

- Set up Docker environment, run full test suite -
- Confirm the is not bug via regression test (should fail on current code) -
- Trace full registration pipeline and lottery initialization flow -
- Finalize coverage targets and is not bug fix strategy with mentors -
- Write technical design doc, get mentor sign-off

- **Coding Phase 1 (May 27 - July 7):**

- Week 1 (May 27-Jun 2):
 - PR1 — Shared test infrastructure
- Week 2 (Jun 3-9):
 - PR2 — ClassCreationController unit tests
- Week 3 (Jun 10-16):
 - PR3 — Lottery controller tests (Part 1: Basic + Invariant)
- Week 4 (Jun 17-23):
 - PR3 — Lottery controller tests (Part 2: Edge + Save)
- Week 5 (Jun 24-30):
 - PR4 — Module handler behavioral tests
- Week 6 (Jul 1-7):
 - PR5 — Student registration invariant tests + buffer

- **Midterm (Jul 7-11):** Components 0-4 complete and tested. All mandatory issue areas (#3780, #3773, #599/#794) addressed. PRs 1-5 submitted or merged.

- **Coding Phase 2 (Jul 14 - Aug 25):**

- Week 1-2:
 - PR6 — Codecov config + documentation
- Week 3:
 - Additional coverage gaps (esp/web views, esp/dbmail/sendto_fns.py)
- Week 4:
 - Full integration testing across multiple program configurations
- Week 5-6:
 - Code review polish, final submission

SECTION 9 : DESIGN SAFETY & BACKWARD COMPATIBILITY

All new code is test code only. No production logic is modified.

- **No Production Code Is Modified**
 - Components 0-4 add test files only. The one exception is codecov.yml (a configuration file) and onsitecheckinmodule.py (adding 3 behavioral tests to an existing test file — no production code). All production modules are read-only from the perspective of this proposal.
- **All Tests Are Isolated and xdist-Safe**
 - All new tests use Django's TestCase (via CacheFlushTestCase or ProgramFrameworkTest), which wraps each test in a transaction rolled back after the test. CacheFlushTestCase rotates the CACHE_PREFIX between tests, ensuring Tag.getProgramTag() and similar cached lookups do not return stale values. No test writes to the filesystem or uses global state.
- **The is not Bug Is Documented, Not Silently Fixed**
 - SetClassDataTest.test_is_not_bug_regression() will FAIL on the current code, confirming the bug exists. The PR documents this as a known bug and proposes the fix (is not → !=). Whether to include the fix in the same PR or a separate PR will be decided with mentors during community bonding. If deferred, the test is marked xfail with a comment explaining the known bug.
- **Selenium Tests Are Opt-In**
 - Selenium tests use the @pytest.mark.selenium marker (already defined in pytest.ini) and are skipped by default. The standard CI run is unchanged.
- **The Codecov Threshold Change Is Non-Breaking**
 - 70% is below the current 69.91%, so this does not immediately fail any existing PR. It only prevents future drops.
- **No Migrations Required**
 - No new models, no new columns, no schema changes of any kind.

SECTION 10 : DELIVERABLES

FEATURE DELIVERABLES:

- D0. Shared Test Infrastructure (Component 0) — PR1
- D1. ClassCreationController Unit Tests (Component 1) — PR2
- D2. Lottery Controller Tests (Component 2) — PR3
- D3. Module Handler Behavioral Tests (Component 3) — PR4
- D4. Student Registration Invariant Tests (Component 4) — PR5
- D5. Codecov Configuration + Documentation — PR6

FILES ADDED (test files only):

- esp/tests/factories.py
- esp/program/modules/tests/support/mixins.py
- esp/program/modules/tests/test_classreg_controller.py
- esp/program/modules/tests/test_studentreg_controller.py
- esp/program/tests/test_lottery.py
- esp/program/modules/tests/test_finaidapprove.py
- esp/program/modules/tests/test_adminreviewapps.py
- esp/program/modules/tests/test_lotterystudentregmodule.py
- docs/dev/testing.rst

FILES MODIFIED:

- esp/program/modules/tests/onsitecheckinmodule.py (add 3 behavioral tests)
- codecov.yml (threshold update)

FILES NOT MODIFIED (by design):

- esp/program/controllers/classreg.py (except possible is not fix)
- esp/program/controllers/lottery.py
- esp/program/modules/handlers/finaidapprovemodule.py
- esp/program/modules/handlers/adminreviewapps.py
- and all other production files

SECTION 11 : STRETCH GOALS & FUTURE EXTENSIONS

STRETCH GOAL 1: Property-Based Testing for Lottery Invariants

- The lottery invariants (no conflicts, no overcapacity, no grade violations) are natural candidates for property-based testing with Hypothesis. Given any valid program configuration, the lottery should always satisfy these invariants. The fixed-seed tests in Component 2 verify specific cases. Hypothesis would verify them for arbitrary inputs — providing stronger guarantees than any fixed test suite can.
- The test infrastructure established by this proposal (`ProgramFrameworkTest + ModuleHandlerTestMixin`) provides the direct foundation. Hypothesis integrates with pytest and Django's TestCase with no additional setup.

STRETCH GOAL 2: Mailman Integration Tests via Mocking

- `esp/mailman` has 30.11% coverage (28/93 lines). The covered lines are imports and the `@enable_with_setting` decorator. Every actual function — `add_list_member()`, `add_list_members()`, `remove_list_member()`, `list_contents()`, `create_list()` — is untested. These are called by `ClassCreationController.associate_teacher_with_class()` and `LotteryAssignmentController.save_assignments()`.
- Since `USE_MAILMAN=False` in the test environment, these calls are no-ops via `@enable_with_setting`. The fix: mock the subprocess calls (`Popen`, `call`) and verify that the correct arguments are passed to the Mailman CLI. This would bring mailman coverage from 30% to ~90% and catch regressions in the `add_list_members()` encoding logic — the iso-8859-1 encoding for non-ASCII names is a known fragile path identified while reading the source.

STRETCH GOAL 3: Performance Regression Tests

- The registration pipeline has no performance tests. A query that was `O(n)` could become `O(n2)` after a model change without any test catching it. `ProgramFrameworkTest` already supports configuring large programs (`num_students=100`, `num_teachers=50`). A lightweight performance test — asserting that the catalog view completes in under 2 seconds for a program with 100 classes — would catch the most egregious regressions without requiring a dedicated performance testing framework.

SECTION 12 : WHY ME?

- I found a production bug in set class data() before writing a single line of this proposal: `k[:8]` is not `'section_'` uses identity comparison, not value comparison. In Python 3, string interning is not guaranteed for sliced strings. This can silently misroute form fields to `cls.__dict__` instead of `custom_form_data`. No existing test catches it. Component 1 includes a regression test that will FAIL on the current code — confirming the bug — and proposes the one-character fix.
- I read `lottery.py` in full: 1000+ lines, 20+ methods, 0 tests. The lottery runs once per program cycle and is irreversible. I designed Component 2 around the hard invariants — no conflicts, no overcapacity, no grade violations — that must hold regardless of the random seed or numpy version. These are the invariants that `check_assignments()` already asserts internally; the tests make them externally verifiable for the first time.
- The infrastructure already exists — `pytest-xdist`, `Codecov`, and `Selenium`. This proposal systematically closes the highest-risk gaps first, converting untested critical paths into enforceable regression guarantees.
- Key design decisions — `CacheFlushTestCase` for Tag tests, invariant-based lottery testing over probabilistic testing, `ModuleHandlerTestMixin` as permanent infrastructure — are derived directly from code behavior, not assumptions.